

在 `local` 变量身上；在成员函数身上，林林总总不一而足。`const` 是个威力强大的助手。尽可能使用它。你会对你的作为感到高兴。

请记住

- 将某些东西声明为 `const` 可帮助编译器侦测出错误用法。`const` 可被施加于任何作用域内的对象、函数参数、函数返回类型、成员函数本体。
- 编译器强制实施 `bitwise constness`，但你编写程序时应该使用“概念上的常量性”（`conceptual constness`）。
- 当 `const` 和 `non-const` 成员函数有着实质等价的实现时，令 `non-const` 版本调用 `const` 版本可避免代码重复。

条款 04：确定对象被使用前已先被初始化

Make sure that objects are initialized before they're used.

关于“将对象初始化”这事，C++ 似乎反复无常。如果你这么写：

```
int x;
```

在某些语境下 `x` 保证被初始化（为 0），但在其他语境中却不保证。如果你这么写：

```
class Point {  
    int x, y;  
};  
...  
Point p;
```

`p` 的成员变量有时候被初始化（为 0），有时候不会。如果你来自其他语言阵营而那儿并不存在“无初值对象”，那么请小心，因为这颇为重要。

读取未初始化的值会导致不明确的行为。在某些平台上，仅仅只是读取未初始化的值，就可能让你的程序终止运行。更可能的情况是读入一些“半随机”bits，污染了正在进行读取动作的那个对象，最终导致不可测知的程序行为，以及许多令人不愉快的调试过程。

现在，我们终于有了一些规则，描述“对象的初始化动作何时一定发生，何时不一定发生”。不幸的是这些规则很复杂，我认为对记忆力而言是太繁复了些。

通常如果你使用 **C part of C++** (见条款 1) 而且初始化可能招致运行期成本, 那么就不保证发生初始化。一旦进入 **non-C parts of C++**, 规则有些变化。这就很好地解释了为什么 **array** (来自 **C part of C++**) 不保证其内容被初始化, 而 **vector** (来自 **STL part of C++**) 却有此保证。

表面上这似乎是个无法决定的状态, 而最佳处理办法就是: 永远在使用对象之前先将它初始化。对于无任何成员的内置类型, 你必须手工完成此事。例如:

```
int x = 0; //对 int 进行手工初始化
const char* text = "A C-style string"; //对指针进行手工初始化
// (亦见条款 3)

double d;
std::cin >> d; //以读取 input stream 的方式完成初始化。
```

至于内置类型以外的任何其他东西, 初始化责任落在构造函数 (**constructors**) 身上。规则很简单: 确保每一个构造函数都将对象的每一个成员初始化。

这个规则很容易奉行, 重要的是别混淆了赋值 (**assignment**) 和初始化 (**initialization**)。考虑一个用来表现通讯簿的 **class**, 其构造函数如下:

```
class PhoneNumber { ... };
class ABEntry { //ABEntry = "Address Book Entry"
public:
    ABEntry(const std::string& name, const std::string& address,
            const std::list<PhoneNumber>& phones);
private:
    std::string theName;
    std::string theAddress;
    std::list<PhoneNumber> thePhones;
    int numTimesConsulted;
};
ABEntry::ABEntry(const std::string& name, const std::string& address,
                const std::list<PhoneNumber>& phones)
{
    theName = name; //这些都是赋值 (assignments),
    theAddress = address; //而非初始化 (initializations)。
    thePhones = phones;
    numTimesConsulted = 0;
}
```

这会导致 `ABEntry` 对象带有你期望（你指定）的值，但不是最佳做法。C++ 规定，对象的成员变量的初始化动作发生在进入构造函数本体之前。在 `ABEntry` 构造函数内，`theName`、`theAddress` 和 `thePhones` 都不是被初始化，而是被赋值。初始化的发生时间更早，发生于这些成员的 *default* 构造函数被自动调用之时（比进入 `ABEntry` 构造函数本体的时间更早）。但这对 `numTimesConsulted` 不为真，因为它属于内置类型，不保证一定在你所看到的那个赋值动作的时间点之前获得初值。

`ABEntry` 构造函数的一个较佳写法是，使用所谓的 **member initialization list**（成员初值列）替换赋值动作：

```
ABEntry::ABEntry(const std::string& name, const std::string& address,
                 const std::list<PhoneNumber>& phones)
    :theName(name),
      theAddress(address),           //现在，这些都是初始化 (initializations)
      thePhones(phones),
      numTimesConsulted(0)
{ }                                  //现在，构造函数本体不必有任何动作
```

这个构造函数和上一个的最终结果相同，但通常效率较高。基于赋值的那个版本（本例第一版本）首先调用 *default* 构造函数为 `theName`、`theAddress` 和 `thePhones` 设初值，然后立刻再对它们赋予新值。*default* 构造函数的一切作为因此浪费了。成员初值列（**member initialization list**）的做法（本例第二版本）避免了这一问题，因为初值列中针对各个成员变量而设的实参，被拿去作为各成员变量之构造函数的实参。本例中的 `theName` 以 `name` 为初值进行 *copy* 构造，`theAddress` 以 `address` 为初值进行 *copy* 构造，`thePhones` 以 `phones` 为初值进行 *copy* 构造。

对大多数类型而言，比起先调用 *default* 构造函数然后再调用 *copy assignment* 操作符，单只调用一次 *copy* 构造函数是比较高效的，有时甚至高效得多。对于内置型对象如 `numTimesConsulted`，其初始化和赋值的成本相同，但为了一致性最好也通过成员初值列来初始化。同样道理，甚至当你想要 *default* 构造一个成员变量，你都可以使用成员初值列，只要指定无物（*nothing*）作为初始化实参即可。假设 `ABEntry` 有一个无参数构造函数，我们可将它实现如下：

```
ABEntry::ABEntry( )
    :theName(),                     //调用 theName 的 default 构造函数;
      theAddress(),                 //为 theAddress 做类似动作;
      thePhones(),                 //为 thePhones 做类似动作;
      numTimesConsulted(0)         //记得将 numTimesConsulted 显式初始化为 0
{ }
```

由于编译器会为用户自定义类型 (user-defined types) 之成员变量自动调用 *default* 构造函数——如果那些成员变量在“成员初值列”中没有被指定初值的话, 因而引发某些程序员过度夸张地采用以上写法。那是可理解的, 但请立下下一个规则, 规定总是在初值列中列出所有成员变量, 以免还得记住哪些成员变量 (如果它们在初值列中被遗漏的话) 可以无需初值。举个例子, 由于 `numTimesConsulted` 属于内置类型, 如果成员初值列 (member initialization list) 遗漏了它, 它就没有初值, 因而可能开启“不明确行为”的潘多拉盒子。

有些情况下即使面对的成员变量属于内置类型 (那么其初始化与赋值的成本相同), 也一定得使用初值列。是的, 如果成员变量是 `const` 或 `references`, 它们就一定需要初值, 不能被赋值 (见条款 5)。为避免需要记住成员变量何时必须在成员初值列中初始化, 何时不需要, 最简单的做法就是: 总是使用成员初值列。这样做有时候绝对必要, 且又往往比赋值更高效。

许多 `classes` 拥有多个构造函数, 每个构造函数有自己的成员初值列。如果这种 `classes` 存在许多成员变量和/或 `base classes`, 多份成员初值列的存在就会导致不受欢迎的重复 (在初值列内) 和无聊的工作 (对程序员而言)。这种情况下可以合理地初值列中遗漏那些“赋值表现像初始化一样好”的成员变量, 改用它们的赋值操作, 并将那些赋值操作移往某个函数 (通常是 `private`), 供所有构造函数调用。这种做法在“成员变量的初值系由文件或数据库读入”时特别有用。然而, 比起经由赋值操作完成的“伪初始化” (pseudo-initialization), 通过成员初值列 (member initialization list) 完成的“真正初始化”通常更加可取。

C++ 有着十分固定的“成员初始化次序”。是的, 次序总是相同: `base classes` 更早于其 `derived classes` 被初始化 (见条款 12), 而 `class` 的成员变量总是以其声明次序被初始化。回头看看 `ABEntry`, 其 `theName` 成员永远最先被初始化, 然后是 `theAddress`, 再来是 `thePhones`, 最后是 `numTimesConsulted`。即使它们在成员初值列中以不同的次序出现 (很不幸那是合法的), 也不会有任何影响。为避免你或你的检阅者迷惑, 并避免某些可能存在的晦涩错误, 当你在成员初值列中条列各个成员时, 最好总是以其声明次序为次序。

译注: 上述所谓晦涩错误, 指的是两个成员变量的初始化带有次序性。例如初始化 `array` 时需要指定大小, 因此代表大小的那个成员变量必须先有初值。

一旦你已经很小心地将“内置型成员变量”明确地加以初始化, 而且也确保你的构造函数运用“成员初值列”初始化 `base classes` 和成员变量, 那就只剩唯一

件事需要操心,就是……呃……深呼吸……“不同编译单元内定义之 non-local static 对象”的初始化次序。

让我们一点一点地探钻这一长串词组。

所谓 static 对象,其寿命从被构造出来直到程序结束为止,因此 stack 和 heap-based 对象都被排除。这种对象包括 global 对象、定义于 namespace 作用域内的对象、在 classes 内、在函数内、以及在 file 作用域内被声明为 static 的对象。函数内的 static 对象称为 local static 对象(因为它们对函数而言是 local),其他 static 对象称为 non-local static 对象。程序结束时 static 对象会被自动销毁,也就是它们的析构函数会在 main() 结束时被自动调用。

所谓编译单元 (translation unit) 是指产出单一目标文件 (single object file) 的那些源码。基本上它是单一源码文件加上其所含入的头文件 (#include files)。

现在,我们关心的问题涉及至少两个源码文件,每一个内含至少一个 non-local static 对象 (也就是说该对象是 global 或位于 namespace 作用域内,抑或在 class 内或 file 作用域内被声明为 static)。真正的问题是:如果某编译单元内的某个 non-local static 对象的初始化动作使用了另一编译单元内的某个 non-local static 对象,它所用到的这个对象可能尚未被初始化,因为 C++ 对“定义于不同编译单元内的 non-local static 对象”的初始化次序并无明确定义。

实例可以帮助理解。假设你有一个 FileSystem class, 它让互联网上的文件看起来好像位于本机 (local)。由于这个 class 使世界看起来像个单一文件系统,你可能会产生一个特殊对象,位于 global 或 namespace 作用域内,象征单一文件系统:

```
class FileSystem {                                //来自你的程序库
public:
    ...
    std::size_t numDisks() const;                //众多成员函数之一
    ...
};
extern FileSystem tfs;                            //预备给客户使用的对象;
                                                //tfs 代表 "the file system"
```

FileSystem 对象绝不是个稀松平常无关痛痒的 (trivial) 对象,因此你的客户如果在 theFileSystem 对象构造完成前就使用它,会得到惨重的灾情。

现在假设某些客户建立了一个 class 用以处理文件系统内的目录 (directories)。很自然他们的 class 会用上 theFileSystem 对象:

```
class Directory {                                //由程序库客户建立
public:
    Directory( params );
    ...
};
Directory::Directory( params )
{
    ...
    std::size_t disks = tfs.numDisks();        //使用 tfs 对象
    ...
}
```

进一步假设, 这些客户决定创建一个 Directory 对象, 用来放置临时文件:

```
Directory tempDir( params );                    //为临时文件而做出的目录
```

现在, 初始化次序的重要性显现出来了: 除非 tfs 在 tempDir 之前先被初始化, 否则 tempDir 的构造函数会用到尚未初始化的 tfs。但 tfs 和 tempDir 是不同的人在不同的时间于不同的源码文件建立起来的, 它们是定义于不同编译单元内的 non-local static 对象。如何能够确定 tfs 会在 tempDir 之前先被初始化?

喔, 你无法确定。再说一次, C++ 对“定义于不同的编译单元内的 non-local static 对象”的初始化相对次序并无明确定义。这是有原因的: 决定它们的初始化次序相当困难, 非常困难, 根本无解。在其最常见形式, 也就是多个编译单元内的 non-local static 对象经由“模板隐式具现化, implicit template instantiations”形成 (而后者自己可能也是经由“模板隐式具现化”形成), 不但不可能决定正确的初始化次序, 甚至往往不值得寻找“可决定正确次序”的特殊情况。

幸运的是, 一个小小的设计便可完全消除这个问题。唯一需要做的是: 将每个 non-local static 对象搬到自己的专属函数内 (该对象在此函数内被声明为 static)。这些函数返回一个 reference 指向它所含的对象。然后用户调用这些函数, 而不直接指涉这些对象。换句话说, non-local static 对象被 local static 对象替换了。Design Patterns 迷哥迷姊们想必认出来了, 这是 **Singleton** 模式的一个常见实现手法。

这个手法的基础在于: C++ 保证, 函数内的 local static 对象会在“该函数被调用期间”“首次遇上该对象之定义式”时被初始化。所以如果你以“函数调用” (返回一个 reference 指向 local static 对象) 替换“直接访问 non-local static 对象”, 你

就获得了保证，保证你所获得的那个 **reference** 将指向一个历经初始化的对象。更棒的是，如果你从未调用 **non-local static** 对象的“仿真函数”，就绝不会引发构造和析构成本；真正的 **non-local static** 对象可没这等便宜！

以此技术施行于 **tfs** 和 **tempDir** 身上，结果如下：

```
class FileSystem { ... };           //同前
FileSystem& tfs()                   //这个函数用来替换 tfs 对象；它在
{                                  //FileSystem class 中可能是个 static。
    static FileSystem fs;          //定义并初始化一个 local static 对象，
    return fs;                    //返回一个 reference 指向上述对象。
}

class Directory { ... };           //同前
Directory::Directory( params )     //同前，但原本的 reference to tfs
{                                  //现在改为 tfs()
    ...
    std::size_t disks = tfs().numDisks( );
    ...
}

Directory& tempDir()               //这个函数用来替换 tempDir 对象；
{                                  //它在 Directory class 中可能是个 static。
    static Directory td;           //定义并初始化 local static 对象，
    return td;                    //返回一个 reference 指向上述对象。
}
```

这么修改之后，这个系统程序的客户完全像以前一样地用它，唯一不同的是他们现在使用 **tfs()** 和 **tempDir()** 而不再是 **tfs** 和 **tempDir**。也就是说他们使用函数返回的“指向 **static** 对象”的 **references**，而不再使用 **static** 对象自身。

这种结构下的 **reference-returning** 函数往往十分单纯：第一行定义并初始化一个 **local static** 对象，第二行返回它。这样的单纯性使它们成为绝佳的 **inlining** 候选人，尤其如果它们被频繁调用的话（见条款 30）。但是从另一个角度看，这些函数“内含 **static** 对象”的事实使它们在多线程系统中带有不确定性。再说一次，任何一种 **non-const static** 对象，不论它是 **local** 或 **non-local**，在多线程环境下“等待某事发生”都会有麻烦。处理这个麻烦的一种做法是：在程序的单线程启动阶段（**single-threaded startup portion**）手工调用所有 **reference-returning** 函数，这可消除与初始化有关的“竞速形势（**race conditions**）”。

当然啦，运用 **reference-returning** 函数防止“初始化次序问题”，前提是其中

有着一个对对象而言合理的初始化次序。如果你有一个系统，其中对象 A 必须在对象 B 之前先初始化，但 A 的初始化能否成功却又受制于 B 是否已初始化，这时候你就有麻烦了。坦白说你自作自受。只要避开如此病态的境况，此处描述的办法应该可以提供你良好的服务，至少在单线程程序中。

既然如此，为避免在对象初始化之前过早地使用它们，你需要做三件事。第一，手工初始化内置型 non-member 对象。第二，使用成员初值列（member initialization lists）对付对象的所有成分。最后，在“初始化次序不确定性”（这对不同编译单元所定义的 non-local static 对象是一种折磨）氛围下加强你的设计。

请记住

- 为内置型对象进行手工初始化，因为 C++ 不保证初始化它们。
- 构造函数最好使用成员初值列（member initialization list），而不要在构造函数本体内使用赋值操作（assignment）。初值列列出的成员变量，其排列次序应该和它们在 class 中的声明次序相同。
- 为免除“跨编译单元之初始化次序”问题，请以 local static 对象替换 non-local static 对象。